Subtopic: Exception Handling

---

1. Definition of Exception

An **exception** is an event that disrupts the normal execution of a program and requires immediate attention by the processor.

- Exceptions are often caused by **errors, unusual conditions, or external events**.
- The processor must **temporarily halt** the current instruction sequence, handle the exception, and then decide whether to resume or terminate execution.

---

2. Types of Exceptions

1. **Interrupts (External Exceptions):**
    - Caused by external signals (I/O devices, timers, user input).
    - Example: Keyboard press, disk I/O completion.
2. **Traps:**
    - Intentional exceptions triggered by a program instruction.
    - Used for system calls and debugging (breakpoints).
3. **Faults:**
    - Detected before an instruction completes.
    - Can usually be corrected, and the instruction restarted.

- Example: Page fault in virtual memory.

4. **Aborts:**
    - Serious unrecoverable errors.
    - Execution cannot be resumed.
    - Example: Hardware failures, parity errors.

---

3. Exception Handling Mechanism

When an exception occurs, the processor follows these steps:

1. **Detect Exception:**
    - The control unit recognizes an unusual condition (e.g., divide by zero).
2. **Save Processor State:**
    - Store Program Counter (PC) and relevant registers so execution can resume.
3. **Transfer Control:**
    - Jump to an **Exception Handler Routine** (a predefined memory address).
4. **Service the Exception:**
    - Execute the handler routine (e.g., perform I/O service, correct error).
5. **Return from Exception:**
    - Restore saved state and resume execution (if possible).

---

4. Exception Handling in Pipelined Processors

In pipelining, exception handling becomes more complex because **multiple instructions are in different stages** simultaneously.

Challenges:

- Must ensure that **precise exceptions** are delivered:
    - All instructions before the faulting instruction are completed.
    - The faulting instruction and subsequent ones are not executed.

Solutions:

- **Pipeline Flush:** Clear instructions after the faulty one.
- **Write Buffering:** Delay register/memory updates until instruction is confirmed safe.

---

5. Importance of Exception Handling

- Ensures **reliability** and **correct execution** of programs.
- Supports **multi-tasking** and **system services** (via traps).
- Provides mechanisms for **error detection, recovery, and debugging**.

---

6. Example Scenarios

1. **Divide by Zero:**

- Instruction `DIV R1, R2` → If R2 = 0, trigger an exception → OS displays error.

2. **Page Fault (Virtual Memory):**
    - Instruction tries to access memory not in RAM → OS brings page from disk → Instruction restarts.
3. **I/O Interrupt:**
    - Disk signals completion → CPU jumps to interrupt handler → Data transferred.

✅ Summary

- Exceptions are events that alter the normal flow of program execution.
- They are classified into **interrupts, traps, faults, and aborts**.
- Exception handling involves **detection, saving state, handling, and resuming/terminating execution**.
- In **pipelined processors**, maintaining **precise exceptions** is crucial, often requiring **pipeline flushes** and **delayed updates**.